

Lec17

Callback Questions

Downloading a File

Problem

Create a function `downloadFile(url, callback)` that:

- Prints "`Starting download...`".
- Waits for 2 seconds.
- Prints "`Download completed`".
- Calls the callback function.

Answer

```
function downloadFile(url, callback) {  
  console.log("Starting download...");
```

```
  setTimeout(() => {  
    console.log("Download completed");  
    callback();  
  }, 2000);  
}
```

```
downloadFile("file.pdf", () => {  
  console.log("Opening file...");  
});
```

User Authentication

Problem

Simulate a login process:

1. Verify user credentials.
2. If successful, fetch user profile.
3. Display profile.

Use nested callbacks.

Answer

```
function login(callback) {  
  setTimeout(() => {  
    console.log("User authenticated");  
    callback();  
  }, 1000);  
}
```

```
function getProfile(callback) {  
  setTimeout(() => {  
    console.log("Profile fetched");  
    callback();  
  }, 1000);  
}
```

```
login(() => {  
  getProfile(() => {  
    console.log("Displaying profile");  
  });  
});
```

Food Delivery Process

Problem

Simulate:

1. Order placed
2. Food prepared
3. Food delivered

using callbacks.

Answer

```
function placeOrder(callback) {  
  setTimeout(() => {  
    console.log("Order placed");  
    callback();  
  }, 1000);  
}
```

```
function prepareFood(callback) {  
  setTimeout(() => {  
    console.log("Food prepared");  
    callback();  
  }, 2000);  
}
```

```
function deliverFood(callback) {  
  setTimeout(() => {  
    console.log("Food delivered");  
    callback();  
  }, 1000);  
}
```

```
placeOrder(() => {  
  prepareFood(() => {  
    deliverFood(() => {  
      console.log("Enjoy your meal!");  
    });  
  });  
});
```

Callback Hell

Problem

Create the following flow:

Start Download

Download Complete

Compressing File

Compression Complete

Uploading File

Upload Complete

Answer

```
function download(cb) {
  setTimeout(() => {
    console.log("Download Complete");
    cb();
  }, 1000);
}

function compress(cb) {
  setTimeout(() => {
    console.log("Compression Complete");
    cb();
  }, 1000);
}

function upload(cb) {
  setTimeout(() => {
    console.log("Upload Complete");
    cb();
  }, 1000);
}

console.log("Start Download");

download(() => {
  console.log("Compressing File");

  compress(() => {
    console.log("Uploading File");

    upload(() => {
      console.log("Process Finished");
    });
  });
});
```

Promise Questions

Download File using Promise

Problem

Convert the download callback example into Promise.

Answer

```
function downloadFile() {  
  return new Promise((resolve) => {  
    console.log("Starting download...");
```

```
    setTimeout(() => {  
      resolve("Download completed");  
    }, 2000);  
  });  
}
```

```
downloadFile()  
  .then(result => {  
    console.log(result);  
  });
```

Check Even or Odd

Problem

Create a Promise that:

- Resolves if a number is even.
- Rejects if odd.

Answer

```
function checkEven(num) {  
  return new Promise((resolve, reject) => {  
    if (num % 2 === 0)  
      resolve("Even Number");  
    else  
      reject("Odd Number");  
  });  
}
```

```
checkEven(8)  
  .then(msg => console.log(msg))  
  .catch(err => console.log(err));
```

ATM Withdrawal

Problem

Create a Promise that:

- Resolves if balance is sufficient.
- Rejects if insufficient.

Answer

```
function withdraw(balance, amount) {  
  return new Promise((resolve, reject) => {  
    if (balance >= amount)  
      resolve("Withdrawal Successful");  
    else  
      reject("Insufficient Balance");  
  });  
}
```

```
withdraw(5000, 2000)  
  .then(msg => console.log(msg))  
  .catch(err => console.log(err));
```

Promise Chaining

Problem

Perform:

1. Download
2. Compress
3. Upload

using Promise chaining.

Answer

```
function download() {
  return new Promise(resolve => {
    setTimeout(() => {
      console.log("Downloaded");
      resolve();
    }, 1000);
  });
}

function compress() {
  return new Promise(resolve => {
    setTimeout(() => {
      console.log("Compressed");
      resolve();
    }, 1000);
  });
}

function upload() {
  return new Promise(resolve => {
    setTimeout(() => {
      console.log("Uploaded");
      resolve();
    }, 1000);
  });
}

download()
  .then(() => compress())
  .then(() => upload())
  .then(() => console.log("All Tasks Completed"));
```

Promise.all()

Problem

Run three API calls simultaneously and wait for all to finish.

Answer

```
const api1 = Promise.resolve("User Data");  
const api2 = Promise.resolve("Orders Data");  
const api3 = Promise.resolve("Payment Data");
```

```
Promise.all([api1, api2, api3])  
  .then(results => {  
    console.log(results);  
  });
```

Task

Implement your own `setTimeout` using callbacks.

Create a login → profile → posts flow using callbacks.

Convert callback hell into Promise chaining.

Build an e-commerce checkout process using Promises.

Implement a traffic light sequence (Red → Yellow → Green) using Promises.

Create a movie ticket booking system using Promises.

Simulate bank transactions using Promises.

Fetch student marks and calculate grade using Promises.

Use `Promise.all()` to load user, posts, and comments together.

Use `Promise.race()` to simulate multiple servers responding.